C. Unstable Elements

time limit per test 2 seconds
memory limit per test 512 megabytes
input standard input
output standard output

Rating: 1300 | Tags: brute force, data structures, implementation, sortings, two pointers |
https://codeforces.com/problemset/problem/2242/C

You are given a **sorted** array of integers $[a_1, a_2, \ldots, a_n]$. We can perform any number of operations of the following type on this array:

- mark the first element of the array, as well as all elements that are not equal to their left neighbors (that is, all elements i such that $a_i \neq a_{i-1}$).
- then either delete all marked elements or duplicate them (that is, replace each marked element with two identical elements).

For example, consider the array [**1**, 1, 1, **2**, **4**, 4, **5**] (marked elements are shown in bold). If we delete all marked elements, we get [1, 1, 4], and if we duplicate them, we get [1, 1, 1, 1, 2, 2, 4, 4, 4, 5, 5].

The operations cannot be performed if the array becomes empty. After each operation, every element gets unmarked.

We call an array of integers b reachable if it can be obtained from array a by some number of the described operations. Your task is to count the number of reachable arrays of length k.

Input

The first line contains one integer t ($1 \le t \le 10^4$) — the number of test cases.

Each test case consists of two lines:

- the first line contains two integers n and k ($1 \le n, k \le 3 \cdot 10^5$);
- the second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_1 \le a_2 \le \ldots \le a_n \le n$). **Note that the array is sorted.**

Additional constraint on the input: the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, output one integer — the number of reachable arrays b of length k. It can be shown that, under the constraints of the problem, the answer fits into a standard 32-bit integer type.

Example

Input

```
10
1 5
1
4 5
1 1 2 2
3 1
1 1 1
4 8
1 2 3 4
8 6
1 1 1 2 2 2 2 2
6 3
1 1 2 2 3 3
10 7
1 1 1 2 2 3 3 3 3 4
12 5
1 1 1 1 2 2 2 3 3 4 4 4
6 5
1 1 2 2 3 3
3 1
1 2 2
```

Output

```
1
0
1
1
2
1
2
1
0
1
```

Note

In the first example, the following sequence of operations can be performed:

- [**1**] → [**1**, 1] → [**1**, 1, 1] → [**1**, 1, 1, 1] → [1, 1, 1, 1, 1].

In the second example, it is impossible to obtain an array of length 5.

In the third example, the following sequence of operations can be performed:

- [**1**, 1, 1] → [**1**, 1] → [1].

In the fifth example, the arrays [1, 1, 2, 2, 2, 2] and [2, 2, 2, 2, 2, 2] can be obtained:

- [**1**, 1, 1, **2**, 2, 2, 2, 2] → [1, 1, 2, 2, 2, 2];
- [**1**, 1, 1, **2**, 2, 2, 2, 2] → [**1**, 1, **2**, 2, 2, 2] → [**1**, **2**, 2, 2] → [**2**, 2, 2] → [**2**, 2, 2, 2] → [**2**, 2, 2, 2, 2] → [2, 2, 2, 2, 2, 2].

In the seventh example, the arrays [1, 1, 1, 3, 3, 3, 3] and [3, 3, 3, 3, 3, 3, 3] can be obtained.